Take Home Lab → Day 1 | Solutions

Task

Go over the following topic →

https://www.geeksforgeeks.org/dsa/little-and-big-endian-mystery/

http://stackoverflow.com/questions/22030657/little-endian-vs-big-endian

Question 1

```
#include <stdio.h>

void doSomething(int *x, int *y) {
  int *temp;
  temp = y;
  y = x;
  x = temp;
}

int main() {

   int a = 2025, b = 0, c = 45, d = 100;
   doSomething(&a, &b);
   if (a < c)
     doSomething(&c, &a);
   doSomething(&a, &d);

   printf("%d", a);

   return 0;
}
```

Without running the code, tell the output of the given program.

💡 **Answer** → 2025

Reason → the function doSomething(...) just swaps the pointers locally, but doesn't actually swap the dereffered value stored at the memory locations x and y are pointing to.

Question 2

```
int main() {

  int a = 300;
  char *b = (char *)&a;
  *++b = 2;

  printf("%d ",a);

  return 0;
}
```

Without running the code, tell the output of the given program.

💡 **Answer** → 556

Reason → The b pointer points to the memory location of a but only holds 1 byte of data. The operation `*++b = 2;` first increments the pointer b and then derefferes the value (1 byte) stored at that location and updates it to 2.

Question 3

Design a Calculator using function pointers in C, which takes the operation, and two values as input. You can implement basic mathematical operators (add, subtract, multiply, divide). Feel free to extend the list.

```
#include <stdio.h>

int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }
int mul(int a, int b) { return a * b; }
int div(int a, int b) { return a / b; }
```

```
int (*ops[])(int, int) = {add, sub, mul, div};

int main() {
    int op = 2; // 0 for add, 1 for sub, 2 for mul, 3 for div
    int val1 = 10;
    int val2 = 3;
    printf("%d\n", ops[op](val1, val2));
    return 0;
}
```

Question 4

```
#include <stdio.h>

int main() {

        unsigned int x = 3251;
        char *cp = &x;
        unsigned char c1 = *cp++, c2 = *cp;
        printf("%d\n",c1);
        printf("%d\n",c2);

        return 0;
}
```

Based on the above question discussed in the class, answer the following questions →

- What is `cp` storing, an address or a value?

  > 💡 cp stores anything that is assigned to it - address or value. If we store an address then we can dereference it.

- What is the size of the memory location occupied by `cp`?

  > 💡 1 byte or 8 bits

- What is the size of `x`?

💡 4 bytes or 32 bits

- What is the size of `&x`?

💡 depends on your OS.

Question 5

💡 As it isn't expected that you know how to pass command line arguments in C right now you can use https://www.onlinegdb.com/online_c_compiler to compile your code using command line arguments for now.

Complete the function to print the command line arguments passed to the program in reversed order.

```
#include <stdio.h>

int main(int argc, char** argv) {

    // Print in reverse order
    printf("Words in reverse:\n");
    for (int i = argc - 1; i > 0; i--) {
        printf("%s\n", *(argv + i));
    }

    return 0;
}
```

Sample Input:

```
Operating Systems is a fun course.
```

Sample Output:

```
course.
fun
```

```
a
is
Systems
Operating
```